JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.stream

Interface DoubleStream

All Superinterfaces:

`AutoCloseable, BaseStream<Double,DoubleStream>`

```
public interface DoubleStream
extends BaseStream<Double,DoubleStream>
```

A sequence of primitive double-valued elements supporting sequential and parallel aggregate operations. This is the `double` primitive specialization of `Stream`.

The following example illustrates an aggregate operation using `Stream` and `DoubleStream`, computing the sum of the weights of the red widgets:

```
double sum = widgets.stream()
                    .filter(w -> w.getColor() == RED)
                    .mapToDouble(w -> w.getWeight())
                    .sum();
```

See the class documentation for `Stream` and the package documentation for java.util.stream for additional specification of streams, stream operations, stream pipelines, and parallelism.

Since:

1.8

See Also:

`Stream, java.util.stream`

Nested Class Summary

Nested Classes	Interface	Description
static interface	DoubleStream.Builder	A mutable builder for a DoubleStream.

Method Summary

Default M

Modifier and Type	Method	Description
boolean	allMatch(DoublePredicate predicate)	Returns wheth elements of th match the prov predicate.
boolean	anyMatch(DoublePredicate predicate)	Returns wheth elements of th match the prov predicate.
OptionalDouble	average()	Returns an OptionalDoub describing the mean of eleme stream, or an optional if this empty.
Stream<Double>	boxed()	Returns a Stre consisting of t of this stream, Double.
static DoubleStream.Builder	builder()	Returns a build DoubleStream.
<R> R	collect(Supplier<R> supplier, ObjDoubleConsumer<R> accumulator, BiConsumer<R,R> combiner)	Performs a mu reduction ope the elements o stream.
static DoubleStream	concat(DoubleStream a, DoubleStream b)	Creates a lazil concatenated s whose element elements of th stream followe

		elements of th stream.
long	**count**()	Returns the co elements in thi
DoubleStream	**distinct**()	Returns a strea consisting of th elements of thi
static **DoubleStream**	**empty**()	Returns an em sequential Dou
DoubleStream	**filter**(**DoublePredicate** predicate)	Returns a strea consisting of th of this stream the given pred
OptionalDouble	**findAny**()	Returns an **OptionalDoub** describing som of the stream, OptionalDoub stream is empt
OptionalDouble	**findFirst**()	Returns an **OptionalDoub** describing the element of this an empty Opti if the stream is
DoubleStream	**flatMap**(**DoubleFunction**<? extends **DoubleStream**> mapper)	Returns a strea consisting of th replacing each this stream wit contents of a m stream produc applying the p mapping funct element.
void	**forEach**(**DoubleConsumer** action)	Performs an ac each element o stream.
void	**forEachOrdered**(**DoubleConsumer** action)	Performs an ac each element o stream, guaran each element i

		in encounter o streams that h defined encour
static **DoubleStream**	**generate**(**DoubleSupplier** s)	Returns an infi sequential uno stream where element is gen the provided DoubleSuppli
static **DoubleStream**	**iterate**(double seed, **DoubleUnaryOperator** f)	Returns an infi sequential ord DoubleStream by iterative ap a function f to element seed, producing a St consisting of s f(seed), f(f(
PrimitiveIterator.OfDouble	**iterator**()	Returns an ite elements of thi
DoubleStream	**limit**(long maxSize)	Returns a strea consisting of th of this stream, to be no longe maxSize in len
DoubleStream	**map**(**DoubleUnaryOperator** mapper)	Returns a strea consisting of th applying the gi function to the this stream.
IntStream	**mapToInt**(**DoubleToIntFunction** mapper)	Returns an Int consisting of th applying the gi function to the this stream.
LongStream	**mapToLong**(**DoubleToLongFunction** mapper)	Returns a Long consisting of th applying the gi function to the this stream.
<U> **Stream**<U>	**mapToObj**(**DoubleFunction**<? extends U> mapper)	Returns an obj Stream consist

		results of appl given function elements of th
OptionalDouble	**max**()	Returns an OptionalDoub describing the element of this an empty Opti if this stream i
OptionalDouble	**min**()	Returns an OptionalDoub describing the element of this an empty Opti if this stream i
boolean	**noneMatch**(**DoublePredicate** predicate)	Returns wheth elements of th match the prov predicate.
static **DoubleStream**	**of**(double... values)	Returns a sequ ordered stream elements are t values.
static **DoubleStream**	**of**(double t)	Returns a sequ DoubleStream single element
DoubleStream	**parallel**()	Returns an equ stream that is
DoubleStream	**peek**(**DoubleConsumer** action)	Returns a strea consisting of th of this stream, performing the action on each elements are c from the result
OptionalDouble	**reduce**(**DoubleBinaryOperator** op)	Performs a **red** the elements o stream, using **associative** ac function, and r OptionalDoub

		describing the value, if any.
double	**reduce**(double identity, **DoubleBinaryOperator** op)	Performs a **red** the elements o stream, using identity value **associative** ac function, and r reduced value.
DoubleStream	**sequential**()	Returns an equ stream that is
DoubleStream	**skip**(long n)	Returns a strea consisting of th remaining elem stream after di the first n elem stream.
DoubleStream	**sorted**()	Returns a strea consisting of th of this stream order.
Spliterator.OfDouble	**spliterator**()	Returns a split the elements o stream.
double	**sum**()	Returns the su elements in thi
DoubleSummaryStatistics	**summaryStatistics**()	Returns a DoubleSummary describing var summary data elements of thi
double[]	**toArray**()	Returns an arr containing the this stream.

Methods inherited from interface java.util.stream.BaseStream

close, isParallel, onClose, unordered

Method Detail

filter

`DoubleStream filter(DoublePredicate predicate)`

Returns a stream consisting of the elements of this stream that match the given predicate.

This is an intermediate operation.

Parameters:

`predicate - a non-interfering, stateless predicate to apply to each element to determine if it should be included`

Returns:

`the new stream`

map

`DoubleStream map(DoubleUnaryOperator mapper)`

Returns a stream consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Parameters:

`mapper - a non-interfering, stateless function to apply to each element`

Returns:

`the new stream`

mapToObj

`<U> Stream<U> mapToObj(DoubleFunction<? extends U> mapper)`

Returns an object-valued `Stream` consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Type Parameters:

`U - the element type of the new stream`

Parameters:

`mapper - a non-interfering, stateless function to apply to each element`

Returns:

the new stream

mapToInt

`IntStream mapToInt(DoubleToIntFunction mapper)`

Returns an `IntStream` consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Parameters:

`mapper - a non-interfering, stateless function to apply to each element`

Returns:

the new stream

mapToLong

`LongStream mapToLong(DoubleToLongFunction mapper)`

Returns a `LongStream` consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Parameters:

`mapper - a non-interfering, stateless function to apply to each element`

Returns:

the new stream

flatMap

`DoubleStream flatMap(DoubleFunction<? extends DoubleStream> mapper)`

Returns a stream consisting of the results of replacing each element of this stream with the contents of a mapped stream produced by applying the provided mapping function to each element. Each mapped stream is `closed` after its contents have been placed into this stream. (If a mapped stream is `null` an empty stream is used, instead.)

This is an intermediate operation.

Parameters:

mapper - a non-interfering, stateless function to apply to each element which produces a DoubleStream of new values

Returns:

the new stream

See Also:

Stream.flatMap(Function)

distinct

DoubleStream distinct()

Returns a stream consisting of the distinct elements of this stream. The elements are compared for equality according to Double.compare(double, double).

This is a stateful intermediate operation.

Returns:

the result stream

sorted

DoubleStream sorted()

Returns a stream consisting of the elements of this stream in sorted order. The elements are compared for equality according to Double.compare(double, double).

This is a stateful intermediate operation.

Returns:

the result stream

peek

DoubleStream peek(DoubleConsumer action)

Returns a stream consisting of the elements of this stream, additionally performing the provided action on each element as elements are consumed from the resulting stream.

This is an intermediate operation.

For parallel stream pipelines, the action may be called at whatever time and in whatever thread the element is made available by the upstream operation.

If the action modifies shared state, it is responsible for providing the required synchronization.

API Note:

This method exists mainly to support debugging, where you want to see the elements as they flow past a certain point in a pipeline:

```
DoubleStream.of(1, 2, 3, 4)
    .filter(e -> e > 2)
    .peek(e -> System.out.println("Filtered value: " + e))
    .map(e -> e * e)
    .peek(e -> System.out.println("Mapped value: " + e))
    .sum();
```

Parameters:

`action` - a non-interfering action to perform on the elements as they are consumed from the stream

Returns:

the new stream

limit

`DoubleStream limit(long maxSize)`

Returns a stream consisting of the elements of this stream, truncated to be no longer than `maxSize` in length.

This is a short-circuiting stateful intermediate operation.

API Note:

While limit() is generally a cheap operation on sequential stream pipelines, it can be quite expensive on ordered parallel pipelines, especially for large values of maxSize, since limit(n) is constrained to return not just any *n* elements, but the *first n* elements in the encounter order. Using an unordered stream source (such as generate(DoubleSupplier)) or removing the ordering constraint with BaseStream.unordered() may result in significant speedups of limit() in parallel pipelines, if the semantics of your situation permit. If consistency with encounter order is required, and you are experiencing poor performance or memory utilization with limit() in parallel pipelines, switching to sequential execution with sequential() may improve performance.

Parameters:

`maxSize` - the number of elements the stream should be limited to

Returns:

the new stream

Throws:

IllegalArgumentException - if maxSize is negative

skip

DoubleStream skip(long n)

Returns a stream consisting of the remaining elements of this stream after discarding the first n elements of the stream. If this stream contains fewer than n elements then an empty stream will be returned.

This is a stateful intermediate operation.

API Note:

While skip() is generally a cheap operation on sequential stream pipelines, it can be quite expensive on ordered parallel pipelines, especially for large values of n, since skip(n) is constrained to skip not just any *n* elements, but the *first n* elements in the encounter order. Using an unordered stream source (such as generate(DoubleSupplier)) or removing the ordering constraint with BaseStream.unordered() may result in significant speedups of skip() in parallel pipelines, if the semantics of your situation permit. If consistency with encounter order is required, and you are experiencing poor performance or memory utilization with skip() in parallel pipelines, switching to sequential execution with sequential() may improve performance.

Parameters:

n - the number of leading elements to skip

Returns:

the new stream

Throws:

IllegalArgumentException - if n is negative

forEach

void forEach(DoubleConsumer action)

Performs an action for each element of this stream.

This is a terminal operation.

For parallel stream pipelines, this operation does *not* guarantee to respect the encounter order of the stream, as doing so would sacrifice the benefit of parallelism. For any given element, the action may be performed at whatever

time and in whatever thread the library chooses. If the action accesses shared state, it is responsible for providing the required synchronization.

Parameters:

`action - a non-interfering action to perform on the elements`

forEachOrdered

```
void forEachOrdered(DoubleConsumer action)
```

Performs an action for each element of this stream, guaranteeing that each element is processed in encounter order for streams that have a defined encounter order.

This is a terminal operation.

Parameters:

`action - a non-interfering action to perform on the elements`

See Also:

`forEach(DoubleConsumer)`

toArray

```
double[] toArray()
```

Returns an array containing the elements of this stream.

This is a terminal operation.

Returns:

`an array containing the elements of this stream`

reduce

```
double reduce(double identity,
              DoubleBinaryOperator op)
```

Performs a reduction on the elements of this stream, using the provided identity value and an associative accumulation function, and returns the reduced value. This is equivalent to:

```
    double result = identity;
    for (double element : this stream)
        result = accumulator.applyAsDouble(result, element)
```

```
    return result;
```

but is not constrained to execute sequentially.

The `identity` value must be an identity for the accumulator function. This means that for all `x`, `accumulator.apply(identity, x)` is equal to `x`. The `accumulator` function must be an associative function.

This is a terminal operation.

API Note:

Sum, min, max, and average are all special cases of reduction. Summing a stream of numbers can be expressed as:

```
    double sum = numbers.reduce(0, (a, b) -> a+b);
```

or more compactly:

```
    double sum = numbers.reduce(0, Double::sum);
```

While this may seem a more roundabout way to perform an aggregation compared to simply mutating a running total in a loop, reduction operations parallelize more gracefully, without needing additional synchronization and with greatly reduced risk of data races.

Parameters:

identity - the identity value for the accumulating function

op - an associative, non-interfering, stateless function for combining two values

Returns:

the result of the reduction

See Also:

sum(), min(), max(), average()

reduce

`OptionalDouble reduce(DoubleBinaryOperator op)`

Performs a reduction on the elements of this stream, using an associative accumulation function, and returns an `OptionalDouble` describing the reduced value, if any. This is equivalent to:

```
boolean foundAny = false;
double result = null;
for (double element : this stream) {
    if (!foundAny) {
        foundAny = true;
        result = element;
    }
    else
        result = accumulator.applyAsDouble(result, element);
}
return foundAny ? OptionalDouble.of(result) : OptionalDouble.empty();
```

but is not constrained to execute sequentially.

The `accumulator` function must be an associative function.

This is a terminal operation.

Parameters:

`op - an associative, non-interfering, stateless function for combining two values`

Returns:

`the result of the reduction`

See Also:

`reduce(double, DoubleBinaryOperator)`

collect

```
<R> R collect(Supplier<R> supplier,
              ObjDoubleConsumer<R> accumulator,
              BiConsumer<R,R> combiner)
```

Performs a mutable reduction operation on the elements of this stream. A mutable reduction is one in which the reduced value is a mutable result container, such as an `ArrayList`, and elements are incorporated by updating the state of the result rather than by replacing the result. This produces a result equivalent to:

```
R result = supplier.get();
for (double element : this stream)
    accumulator.accept(result, element);
return result;
```

Like `reduce(double, DoubleBinaryOperator)`, `collect` operations can be parallelized without requiring additional synchronization.

This is a terminal operation.

Type Parameters:

`R - type of the result`

Parameters:

`supplier - a function that creates a new result container. For a parallel execution, this function may be called multiple times and must return a fresh value each time.`

`accumulator - an associative, non-interfering, stateless function for incorporating an additional element into a result`

`combiner - an associative, non-interfering, stateless function for combining two values, which must be compatible with the accumulator function`

Returns:

`the result of the reduction`

See Also:

`Stream.collect(Supplier, BiConsumer, BiConsumer)`

sum

```
double sum()
```

Returns the sum of elements in this stream. Summation is a special case of a reduction. If floating-point summation were exact, this method would be equivalent to:

```
     return reduce(0, Double::sum);
```

However, since floating-point summation is not exact, the above code is not necessarily equivalent to the summation computation done by this method.

If any stream element is a NaN or the sum is at any point a NaN then the sum will be NaN. The value of a floating-point sum is a function both of the input values as well as the order of addition operations. The order of addition operations of this method is intentionally not defined to allow for implementation flexibility to improve the speed and accuracy of the computed result. In particular, this method may be implemented using compensated summation or other technique to reduce the error bound in the numerical sum compared to a simple summation of `double` values.

This is a terminal operation.

API Note:

Elements sorted by increasing absolute magnitude tend to yield more accurate results.

Returns:

the sum of elements in this stream

min

`OptionalDouble min()`

Returns an `OptionalDouble` describing the minimum element of this stream, or an empty OptionalDouble if this stream is empty. The minimum element will be `Double.NaN` if any stream element was NaN. Unlike the numerical comparison operators, this method considers negative zero to be strictly smaller than positive zero. This is a special case of a reduction and is equivalent to:

```
return reduce(Double::min);
```

This is a terminal operation.

Returns:

an OptionalDouble containing the minimum element of this stream, or an empty optional if the stream is empty

max

`OptionalDouble max()`

Returns an `OptionalDouble` describing the maximum element of this stream, or an empty OptionalDouble if this stream is empty. The maximum element will be `Double.NaN` if any stream element was NaN. Unlike the numerical comparison operators, this method considers negative zero to be strictly smaller than positive zero. This is a special case of a reduction and is equivalent to:

```
return reduce(Double::max);
```

This is a terminal operation.

Returns:

an OptionalDouble containing the maximum element of this stream, or an empty optional if the stream is empty

count

`long count()`

Returns the count of elements in this stream. This is a special case of a reduction and is equivalent to:

```
return mapToLong(e -> 1L).sum();
```

This is a terminal operation.

Returns:

the count of elements in this stream

average

`OptionalDouble average()`

Returns an `OptionalDouble` describing the arithmetic mean of elements of this stream, or an empty optional if this stream is empty. If any recorded value is a NaN or the sum is at any point a NaN then the average will be NaN.

The average returned can vary depending upon the order in which values are recorded. This method may be implemented using compensated summation

or other technique to reduce the error bound in the `numerical sum` used to compute the average.

The average is a special case of a reduction.

This is a terminal operation.

API Note:

`Elements sorted by increasing absolute magnitude tend to yield more accurate results.`

Returns:

`an OptionalDouble containing the average element of this stream, or an empty optional if the stream is empty`

summaryStatistics

`DoubleSummaryStatistics summaryStatistics()`

Returns a `DoubleSummaryStatistics` describing various summary data about the elements of this stream. This is a special case of a reduction.

This is a terminal operation.

Returns:

`a DoubleSummaryStatistics describing various summary data about the elements of this stream`

anyMatch

`boolean anyMatch(DoublePredicate predicate)`

Returns whether any elements of this stream match the provided predicate. May not evaluate the predicate on all elements if not necessary for determining the result. If the stream is empty then `false` is returned and the predicate is not evaluated.

This is a short-circuiting terminal operation.

API Note:

`This method evaluates the existential quantification of the predicate over the elements of the stream (for some x P(x)).`

Parameters:

`predicate - a non-interfering, stateless predicate to apply to elements of this stream`

Returns:

true if any elements of the stream match the provided predicate, otherwise false

allMatch

boolean allMatch(DoublePredicate predicate)

Returns whether all elements of this stream match the provided predicate. May not evaluate the predicate on all elements if not necessary for determining the result. If the stream is empty then true is returned and the predicate is not evaluated.

This is a short-circuiting terminal operation.

API Note:

This method evaluates the *universal quantification* of the predicate over the elements of the stream (for all x P(x)). If the stream is empty, the quantification is said to be *vacuously satisfied* and is always true (regardless of P(x)).

Parameters:

predicate - a non-interfering, stateless predicate to apply to elements of this stream

Returns:

true if either all elements of the stream match the provided predicate or the stream is empty, otherwise false

noneMatch

boolean noneMatch(DoublePredicate predicate)

Returns whether no elements of this stream match the provided predicate. May not evaluate the predicate on all elements if not necessary for determining the result. If the stream is empty then true is returned and the predicate is not evaluated.

This is a short-circuiting terminal operation.

API Note:

This method evaluates the *universal quantification* of the negated predicate over the elements of the stream (for all x ~P(x)). If the stream is empty, the quantification is said to be vacuously satisfied and is always true, regardless of P(x).

Parameters:

predicate - a non-interfering, stateless predicate to apply to elements of this stream

Returns:

`true if either no elements of the stream match the provided predicate or the stream is empty, otherwise false`

findFirst

`OptionalDouble findFirst()`

Returns an `OptionalDouble` describing the first element of this stream, or an empty `OptionalDouble` if the stream is empty. If the stream has no encounter order, then any element may be returned.

This is a short-circuiting terminal operation.

Returns:

`an OptionalDouble describing the first element of this stream, or an empty OptionalDouble if the stream is empty`

findAny

`OptionalDouble findAny()`

Returns an `OptionalDouble` describing some element of the stream, or an empty `OptionalDouble` if the stream is empty.

This is a short-circuiting terminal operation.

The behavior of this operation is explicitly nondeterministic; it is free to select any element in the stream. This is to allow for maximal performance in parallel operations; the cost is that multiple invocations on the same source may not return the same result. (If a stable result is desired, use `findFirst()` instead.)

Returns:

`an OptionalDouble describing some element of this stream, or an empty OptionalDouble if the stream is empty`

See Also:

`findFirst()`

boxed

`Stream<Double> boxed()`

Returns a `Stream` consisting of the elements of this stream, boxed to `Double`.

This is an intermediate operation.

Returns:

a Stream consistent of the elements of this stream, each boxed to a Double

sequential

```
DoubleStream sequential()
```

Description copied from interface: BaseStream

Returns an equivalent stream that is sequential. May return itself, either because the stream was already sequential, or because the underlying stream state was modified to be sequential.

This is an intermediate operation.

Specified by:

sequential in interface BaseStream<Double,DoubleStream>

Returns:

a sequential stream

parallel

```
DoubleStream parallel()
```

Description copied from interface: BaseStream

Returns an equivalent stream that is parallel. May return itself, either because the stream was already parallel, or because the underlying stream state was modified to be parallel.

This is an intermediate operation.

Specified by:

parallel in interface BaseStream<Double,DoubleStream>

Returns:

a parallel stream

iterator

```
PrimitiveIterator.OfDouble iterator()
```

Description copied from interface: BaseStream

Returns an iterator for the elements of this stream.

This is a terminal operation.

Specified by:

`iterator` in interface `BaseStream<Double,DoubleStream>`

Returns:

the element iterator for this stream

spliterator

`Spliterator.OfDouble spliterator()`

Description copied from interface: `BaseStream`

Returns a spliterator for the elements of this stream.

This is a terminal operation.

Specified by:

`spliterator` in interface `BaseStream<Double,DoubleStream>`

Returns:

the element spliterator for this stream

builder

`static DoubleStream.Builder builder()`

Returns a builder for a `DoubleStream`.

Returns:

a stream builder

empty

`static DoubleStream empty()`

Returns an empty sequential `DoubleStream`.

Returns:

an empty sequential stream

of

`static DoubleStream of(double t)`

Returns a sequential `DoubleStream` containing a single element.

Parameters:

t - the single element

Returns:

a singleton sequential stream

of

```
static DoubleStream of(double... values)
```

Returns a sequential ordered stream whose elements are the specified values.

Parameters:

values - the elements of the new stream

Returns:

the new stream

iterate

```
static DoubleStream iterate(double seed,
                            DoubleUnaryOperator f)
```

Returns an infinite sequential ordered `DoubleStream` produced by iterative application of a function `f` to an initial element `seed`, producing a `Stream` consisting of `seed`, `f(seed)`, `f(f(seed))`, etc.

The first element (position `0`) in the `DoubleStream` will be the provided `seed`. For `n > 0`, the element at position `n`, will be the result of applying the function `f` to the element at position `n - 1`.

Parameters:

seed - the initial element

f - a function to be applied to the previous element to produce a new element

Returns:

a new sequential DoubleStream

generate

```
static DoubleStream generate(DoubleSupplier s)
```

Returns an infinite sequential unordered stream where each element is generated by the provided `DoubleSupplier`. This is suitable for generating constant streams, streams of random elements, etc.

Parameters:

s - the DoubleSupplier for generated elements

Returns:

a new infinite sequential unordered DoubleStream

concat

```
static DoubleStream concat(DoubleStream a,
                           DoubleStream b)
```

Creates a lazily concatenated stream whose elements are all the elements of the first stream followed by all the elements of the second stream. The resulting stream is ordered if both of the input streams are ordered, and parallel if either of the input streams is parallel. When the resulting stream is closed, the close handlers for both input streams are invoked.

Implementation Note:

Use caution when constructing streams from repeated concatenation. Accessing an element of a deeply concatenated stream can result in deep call chains, or even StackOverflowException.

Parameters:

a - the first stream

b - the second stream

Returns:

the concatenation of the two input streams

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.